UNIVERSIDAD POLITÉCNICA SALESIANA

PROGRAMACIÓN EN JAVA · DEBER 1

Sistema de control de un parqueadero

Ingreso de vehículos, registro de salidas y cobro por hora o fracción

Dato	Detalle
Estudiante	Andrés Sisa
Lenguaje y entorno de destino	Java 17 · Eclipse IDE
Interfaz	Consola
Fecha	18 de octubre de 2026
Tarifa solicitada	USD 0,50 por hora o fracción
Persistencia	vehiculos.txt y caja.txt

Resumen

Se implementó una aplicación orientada a objetos para registrar visitas al parqueadero, calcular el valor de cada salida y conservar el historial en archivos de texto. El trabajo sigue las cuatro etapas de la consigna: análisis, diseño, construcción y validación. Se ejecutaron 30 casos de lógica y persistencia y 7 casos de consola, todos aprobados. El proyecto incluye configuración de Eclipse, código comentado y commits Git reales por funcionalidad.

La validación se realizó con Java en Linux. El código y sus commits por funcionalidad están publicados en GitHub. El proyecto está preparado para importarse y ejecutarse en Eclipse.

Contenido: análisis (2), diseño (3-4), construcción (5), validación (6-7), evidencias (8), Eclipse y Git (9), conclusiones y referencias (10).

01 · ANÁLISIS

Problema, entradas y resultados

Objetivo general

Desarrollar una aplicación Java que controle el ingreso y la salida de vehículos de un parqueadero, determine el valor a pagar a razón de USD 0,50 por hora o fracción y almacene la información en los archivos indicados.

Requisitos y criterios de aceptación

ID	Requisito	Cómo se verifica
R1	Registrar placa, fecha y hora de ingreso.	El vehículo aparece dentro y permanece al reiniciar.
R2	Registrar salida de un vehículo activo.	Se cierra la visita y se muestra su comprobante.
R3	Cobrar USD 0,50 por hora iniciada.	60 min = 0,50; 60 min y 1 s = 1,00.
R4	Guardar datos en vehiculos.txt y caja.txt.	Los TXT se crean y conservan información legible.
R5	Organizar el código mediante clases.	Se separan interfaz, negocio, datos y cálculo.
R6	Validar con caja negra y caja blanca.	Casos, resultados y rutas documentados.
R7	Entregar informe y commits importantes.	PDF incluido y commits publicados en GitHub.

Entradas, procesos y salidas

Entradas	Procesos	Salidas
Opción del menú y placa.	Normalizar placa y buscar visita activa.	Confirmación o mensaje de error.
Fecha y hora de entrada/salida.	Validar orden temporal; calcular duración y horas.	Comprobante y valor a pagar.
Historial guardado.	Leer visitas y acumular importes.	Vehículos dentro, historial y total de caja.

Alcance

Se elige consola, una de las alternativas que aparecen en la pizarra. La consigna no exige cupo máximo, tarifas diferenciadas ni interfaz gráfica. La solución agrega consultas y validaciones para poder demostrar el funcionamiento completo. La caja representa el total acumulado de visitas finalizadas, sin cierre diario.

02 · DISEÑO

Diagrama de clases

La aplicación contiene seis clases principales. Vehiculo representa una visita, por lo que una misma placa puede aparecer varias veces en el historial después de completar sus salidas. Cada visita dispone de un identificador independiente.

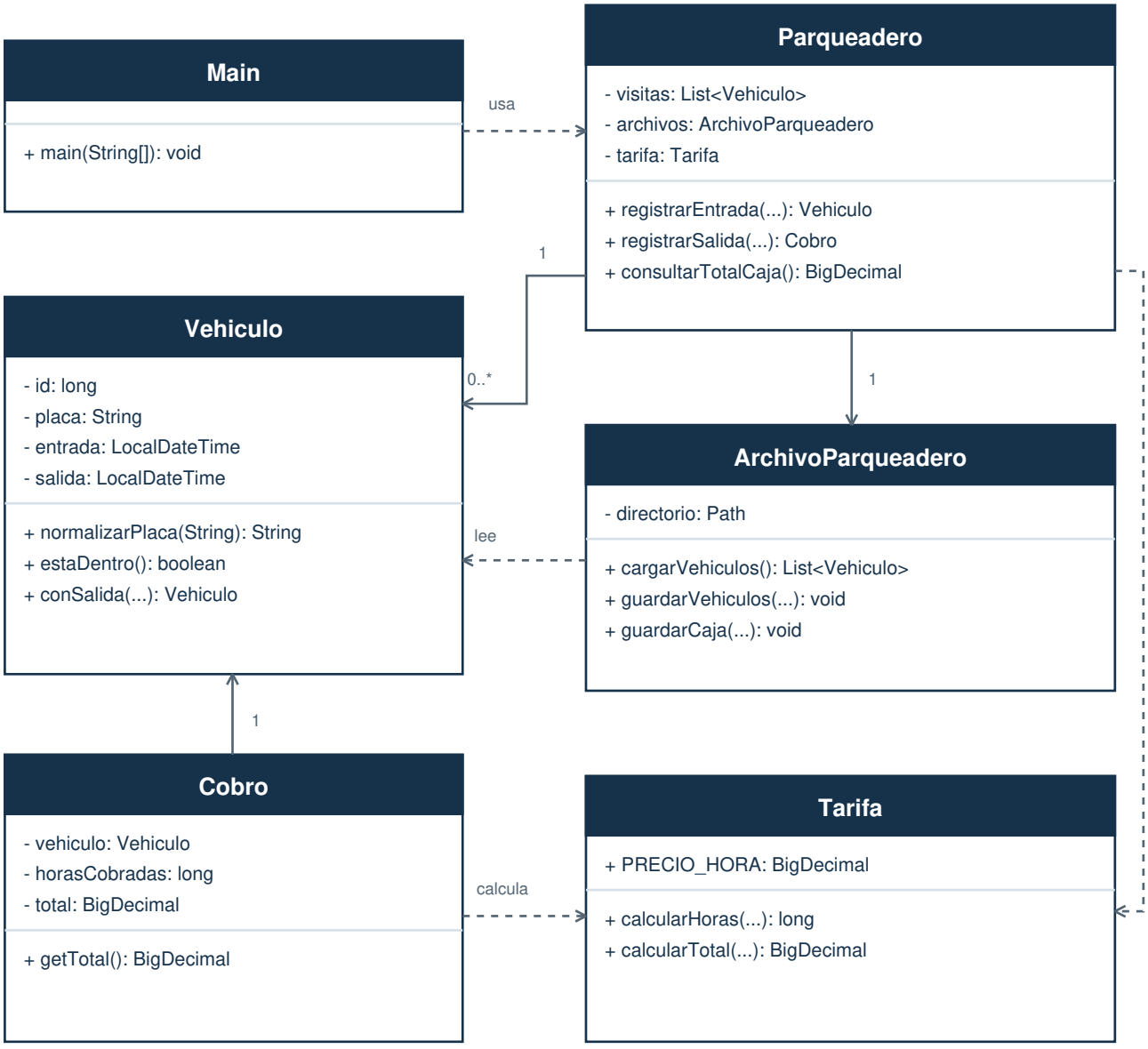


Figura 1. Relaciones principales y miembros relevantes. + indica acceso público; - indica acceso privado. Las líneas discontinuas representan dependencias. Se omiten constructores, getters y algunos parámetros para facilitar la lectura. El archivo diagrama_clases.mmd contiene la versión editable. Fuente: elaboración del proyecto.

Distribución de responsabilidades

Main interactúa con la consola; **Parqueadero** aplica las reglas; **ArchivoParqueadero** lee y escribe; **Vehiculo** conserva los datos de la visita; **Tarifa** calcula horas e importe; **Cobro** reúne el comprobante. Esta separación permite probar el negocio sin utilizar el menú.

02 · DISEÑO

Reglas, archivos y secuencia de salida

Regla de negocio

Para una duración positiva, el número de horas cobradas corresponde al redondeo hacia arriba de la duración expresada en horas. El importe es ese número multiplicado por 0,50. Se consideran segundos y fracciones de segundo para no perder una fracción después de una hora exacta.

```
horas_cobradas = techo(duracion / una_hora)
total_usd = horas_cobradas * 0.50
```

Supuestos explícitos: duración cero genera USD 0,00; una salida anterior se rechaza. Las fechas son locales al equipo. La placa acepta de 5 a 10 letras/números, con al menos una letra y un número; se eliminan guiones y se convierten letras a mayúsculas. Esta validación es del ejercicio, no una verificación oficial de matrícula.

Diseño de los archivos de texto

Archivo	Campos y significado
vehiculos.txt	id;placa;entrada;salida Una fila por visita; salida vacía equivale a vehículo dentro.
caja.txt	id;placa;entrada;salida;horas_cobradas;total_usd Una fila por visita finalizada; importe decimal en USD.

Los campos se separan con punto y coma; se escribe en UTF-8. Las fechas del archivo usan ISO, por ejemplo 2026-10-18T08:00. El historial es la fuente principal; caja se genera a partir de las visitas cerradas. Así, un reinicio no agrega cobros duplicados.

Secuencia del método registrarSalida

Paso	Acción
1	Normalizar la placa y buscar una visita activa. Si no existe, rechazar.
2	Validar que la salida no sea anterior a la entrada.
3	Crear la visita cerrada; calcular las horas y el importe.
4	Guardar el nuevo historial antes de cambiar el estado en memoria.
5	Actualizar caja y devolver el comprobante. Si caja falla, avisar y conservar la salida registrada.

La escritura usa un temporal y luego reemplaza el archivo de destino. Si el sistema no admite movimiento atómico, se usa reemplazo normal. No existe una transacción atómica entre ambos TXT; la reconstrucción de caja permite recuperarla desde el historial.

03 · CONSTRUCCIÓN

Implementación y métodos importantes

El proyecto usa Java 17 sin dependencias externas de ejecución. Se incluyen .project, .classpath, preferencias UTF-8 y una configuración de lanzamiento para Eclipse. Los paquetes son ec.edu.ups.parqueadero; src contiene el programa y test contiene las pruebas.

Cálculo de horas iniciadas

```
Duration tiempo = Duration.between(entrada, salida);
if (tiempo.isNegative()) {
    throw new IllegalArgumentException(
        "La salida no puede ser anterior a la entrada.");
}
long segundos = tiempo.getSeconds();
long horas = segundos / 3600;
if (segundos % 3600 != 0 || tiempo.getNano() != 0) {
    horas++;
}
return horas;
```

El fragmento corresponde a la lógica de calcularHoras. Duration permite representar intervalos con segundos y nanosegundos (Oracle, s. f.-b). El importe usa BigDecimal creado desde la cadena "0.50", multiplicado por horas enteras, para conservar exactitud decimal (Oracle, s. f.-a).

Métodos que organizan la solución

Método	Responsabilidad
registrarEntrada	Evita placas activas duplicadas, asigna ID y guarda una nueva visita.
registrarSalida	Busca la visita activa, valida el tiempo, guarda su salida y devuelve Cobro.
calcularHoras / calcularTotal	Aplica la regla de hora o fracción y calcula el importe.
cargarVehiculos	Lee el historial y detecta líneas inválidas, IDs repetidos y visitas superpuestas.
guardarVehiculos / guardarCaja	Escribe los dos archivos con su formato definido.
consultarTotalCaja	Suma los importes de las visitas finalizadas.

Manejo de errores

El menú controla opciones inválidas, fechas imposibles, placas incorrectas y errores de archivo. Si la entrada de consola termina a mitad de un registro, se cancela la operación. Las listas devueltas al exterior son inmodificables para impedir que se altere el historial sin guardar.

04 · VALIDACIÓN

Pruebas de caja negra

Se comparan entradas y resultados observables. Las fechas base son 18/10/2026 a las 08:00:00; se cambia la salida según cada caso. Las pruebas usan directorios temporales independientes.

Caso	Entrada o condición	Esperado / obtenido
T01	Misma fecha de entrada y salida.	USD 0,00 / 0,00
T02-T03	1 segundo; 59 min 59 s.	USD 0,50 / 0,50
T04	60 minutos exactos.	USD 0,50 / 0,50
T05	60 minutos y 1 segundo.	USD 1,00 / 1,00
T06-T07	2 horas; 24 horas.	USD 1,00 y 12,00 / iguales
T08	23:50 a 00:10 del día siguiente.	USD 0,50 / 0,50
T10, T17	Salida anterior a la entrada.	Rechazo sin cambiar estado / OK
T14	Ingreso abc1234 con ABC-1234 dentro.	Duplicado rechazado / OK
T16, T18	Placa desconocida; segunda salida.	Sin cobro adicional / OK
T19-T21	Reingreso, reinicio y caja eliminada.	Recuperación sin duplicados / OK
T30	Tres visitas de 20 minutos.	Caja USD 1,50 / 1,50

Verificación del menú en procesos Java reales

Caso	Escenario y resultado
U01	Entrada, listado, salida de 61 minutos y caja: USD 1,00.
U02	Cerrar y volver a abrir: activo y caja conservados.
U03	Opción no numérica y fecha 30 de febrero: errores controlados.
U04	Fecha vacía: usa la hora actual.
U05	Fin de entrada a mitad de operación: cancelación sin guardar.
U06	Duplicado, salida desconocida y fecha anterior: caja inalterada.
U07	Historial con cabecera inválida: diagnóstico y salida con código 1.

Resultado: 7/7 escenarios de consola aprobados. El registro completo se entrega en docs/evidencias/pruebas-consola.txt.

04 · VALIDACIÓN

Pruebas de caja blanca

Los casos también se seleccionan según las decisiones internas del código. La tabla vincula condiciones con pruebas ejecutadas; no constituye una medición automática del porcentaje de cobertura.

Método o decisión	Rutas verificadas	Casos
Tarifa.calcularHoras	Duración negativa; cero; hora exacta; resto de segundos; nanosegundo adicional.	T01, T02, T04, T09, T10
registrarEntrada	Placa nueva; duplicada activa; reingreso válido; superposición histórica.	T13, T14, T19, T26
registrarSalida	Visita activa; fecha anterior; placa ausente; visita ya cerrada.	T15-T18
cargarVehiculos	Archivo nuevo; existente; línea inválida; ID repetido; dos visitas activas.	T12, T13, T22, T27, T28
guardarCambios / actualizarCaja	Guardado correcto; fallo del historial; fallo de caja y recuperación.	T20, T21, T23, T24
Protección de datos	Historial ausente con caja existente; listas inmodificables; suma exacta.	T25, T29, T30

Resultado reproducible

Se compiló el código con advertencias de javac activadas y se ejecutó PruebasParqueadero. El ejecutor usa comprobaciones que lanzan AssertionError si el resultado difiere; no depende de habilitar la opción -ea. Los temporales de las pruebas se eliminan al finalizar.

```
PRUEBAS PARQUEADERO | Java 17.0.20
[OK] T23 Fallo de caja avisa y permite recuperarla
[OK] T24 Fallo al guardar historial no cambia memoria
[OK] T25 No borra caja si falta un historial con cobros
[OK] T26 Impide reingreso anterior a una salida previa
[OK] T27 Rechaza identificadores repetidos en archivo
[OK] T28 Rechaza visitas activas duplicadas en archivo
[OK] T29 No expone listas modificables
[OK] T30 Suma tres cobros con exactitud decimal
RESULTADO: 30/30 pruebas aprobadas.
```

Extracto del registro real docs/evidencias/pruebas-java.txt. Se omiten los casos iniciales en esta página; el archivo conserva todos los resultados.

Límites de la validación

Se verificaron los casos descritos con OpenJDK 17.0.20 en Linux. No se ejecutó la interfaz de Eclipse ni los scripts de Windows en este entorno. No se midió cobertura con JaCoCo ni se simularon todas las fallas posibles de disco; tampoco se validó uso simultáneo por varios procesos.

04 · EVIDENCIAS

Ejecución y archivos obtenidos

La demostración ingresó ABC-1234 a las 08:00 y XYZ-5678 a las 09:00; luego registró la salida de ABC-1234 a las 09:01. El segundo vehículo permaneció dentro. Estas son transcripciones de la salida real del programa, no capturas de Eclipse.

Comprobante y consulta de caja

```
COMPROBANTE | ID 1 | ABC1234
Entrada: 2026-10-18 08:00:00
Salida:  2026-10-18 09:01:00
Horas cobradas: 2
TOTAL A PAGAR: USD 1.00

ID 1 | ABC1234 | 2 hora(s) | USD 1.00
Cobros registrados: 1
TOTAL CAJA: USD 1.00
```

Contenido de vehiculos.txt

```
id;placa;entrada;salida
1;ABC1234;2026-10-18T08:00;2026-10-18T09:01
2;XYZ5678;2026-10-18T09:00;
```

Contenido de caja.txt

```
id;placa;entrada;salida;horas_cobradas;total_usd
1;ABC1234;2026-10-18T08:00;2026-10-18T09:01;2;1.00
```

La visita 1 conserva la salida y genera un cobro de USD 1,00; la visita 2 mantiene la salida vacía. Después de terminar y volver a iniciar el proceso, se comprobó que XYZ5678 seguía activo y el total de caja permanecía en USD 1,00.

Dónde encontrar las evidencias

Archivo	Contenido
entrada-demo.txt	Secuencia de entradas utilizada para la demostración.
salida-demo.txt	Salida íntegra de la consola, incluido el menú.
pruebas-java.txt	30 resultados de lógica y persistencia.
pruebas-consola.txt	7 resultados del menú mediante procesos Java.

Los TXT de ejemplo están en docs/ejemplos. El programa crea sus propios archivos de trabajo en la carpeta de ejecución, sin cargar automáticamente esos ejemplos.

05 · ENTREGA

Ejecución en Eclipse y commits

Importar y ejecutar

1. Descomprimir el ZIP y abrir Eclipse con un JDK 17 configurado.
2. Elegir **File > Import > General > Existing Projects into Workspace**.
3. Seleccionar como raíz la carpeta ParqueaderoJava y finalizar.
4. Abrir Main.java y usar **Run As > Java Application**.
5. Escribir en Console; pulsar F5 para visualizar los TXT creados.

El asistente de importación permite incorporar proyectos existentes, y el lanzamiento Java ejecuta su método main (Eclipse Foundation, s. f.-a, s. f.-b). El archivo ParqueaderoJava.launch fija el directorio de trabajo. Para probar, ejecutar PruebasParqueadero.java de la misma forma.

Historial real por funciones importantes

Commit	Cambio registrado
0b33ea1	Inicialización del repositorio remoto.
e673a50	Proyecto Eclipse y modelo Vehiculo con validaciones.
f1a7755	calcularHoras, calcularTotal y comprobante Cobro.
cbf2fdb	Carga del historial y escritura de ambos TXT.
4eef92b	registrarEntrada y listarActivos, sin duplicados.
0a01e67	registrarSalida y consultarTotalCaja.
5f6ca56	Menú de consola, scripts y configuración Eclipse.
946a81e	Pruebas y evidencias de ejecución.

Estos identificadores corresponden a commits publicados en GitHub. Un noveno commit incorpora este informe y las guías; su identificador se consulta con git log. La solución se desarrolló con asistencia de Codex. El código puede revisarse con git show <hash>. Git registra versiones de los cambios confirmados (Chacon & Straub, 2014).

Repositorio publicado

El repositorio público es https://github.com/sisaa5436-arch/ParqueaderoJava.
El punto al final forma parte de su nombre. La rama main conserva las etapas de implementación y sus pruebas. El historial se respalda también en ParqueaderoJava.bundle.

```
git clone https://github.com/sisaa5436-arch/ParqueaderoJava..git ParqueaderoJava
cd ParqueaderoJava
git log --oneline --reverse
```

Los identificadores remotos difieren de los commits de la primera entrega local porque las etapas se publicaron mediante la integración de GitHub. La versión actual del ZIP y del bundle utiliza el historial publicado.

06 · CIERRE

Conclusiones y referencias

Conclusiones

La solución implementa las operaciones de ingreso y salida, calcula el precio por cada hora iniciada y mantiene los datos en los dos archivos solicitados. La separación en clases permite ubicar con claridad las reglas del negocio, el cálculo del importe y la persistencia. Los resultados de 30 casos de lógica/archivos y 7 escenarios del menú respaldan el funcionamiento para las condiciones evaluadas.

El historial de visitas permite recuperar el estado al reiniciar y reconstruir caja sin duplicar cobros. Las validaciones rechazan placas activas repetidas, salidas anteriores y datos malformados. El código y los commits ya se encuentran publicados. La revisión y ejecución en Eclipse permiten al estudiante comprobar la instalación en su equipo y preparar la explicación de los métodos.

Limitaciones y mejoras posibles

El alcance es académico y de una sola instancia. Se reescribe el historial completo en cada cambio; no hay bloqueo entre procesos. La tarifa es fija: modificarla recalcularía la caja histórica. Una ampliación podría guardar la tarifa aplicada por visita, incorporar control de capacidad e implementar persistencia transaccional para uso concurrente.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository

Eclipse Foundation. (s. f.-a). *Importing existing projects*. Eclipse Help. https://help.eclipse.org/latest/topic/org.eclipse.platform.doc.user/tasks/tasks-importproject.htm

Eclipse Foundation. (s. f.-b). *Launching a Java program*. Eclipse Help. https://help.eclipse.org/latest/topic/org.eclipse.jdt.doc.user/tasks/task-launching_java_program.htm

Oracle. (s. f.-a). *BigDecimal (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigDecimal.html

Oracle. (s. f.-b). *Duration (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/Duration.html

Fuentes oficiales consultadas el 18 de octubre de 2026. La consigna se tomó de la fotografía proporcionada por el estudiante. Los diagramas, el código y los registros de prueba corresponden a esta entrega.